



ORTA DOĞU TEKNİK ÜNİVERSİTESİ
MIDDLE EAST TECHNICAL UNIVERSITY
KUZEY KIBRIS KAMPUSU ♦ NORTHERN CYPRUS CAMPUS

CNG 495
Fall – 2024
Final Report

Name: Umutcan

Surname: CELIK

SID: 2526200

Project Title: ZeroDay Project

Date of Submission: 12.15.2024

Table of Contents

Project Description.....	4
Cloud Delivery Models.....	4
<i>SaaS (Software as a Service):.....</i>	<i>4</i>
<i>PaaS (Platform as a Service):</i>	<i>4</i>
<i>IaaS (Infrastructure as a Service):</i>	<i>4</i>
Diagrams	5
<i>Use Case Diagram</i>	<i>5</i>
<i>Data Flow Diagram</i>	<i>5</i>
Data Types.....	6
Computation	7
Expected Contribution.....	7
Releated Work	7
Novelties of the Project	7
Contribution to the Field	8
Milestones Achieved	8
<i>Week 4 (Oct 21 - 27) -- Week 5 (Oct 28 - Nov 3).....</i>	<i>8</i>
<i>Week 6 (Nov 4 - 10)</i>	<i>10</i>
<i>Week 7 (Nov 11 - 17) -- Week 8 (Nov 18 - 24)</i>	<i>12</i>
<i>Week 9 (Nov 25 - Dec 1).....</i>	<i>16</i>
<i>Week 10 (Dec 2 - 8)</i>	<i>20</i>
<i>Week 11 (Dec 9 – Dec 15)</i>	<i>23</i>
References	28

List of Figures

Figure 1: use case diagram of project.....	5
Figure 2: context level of project.....	5
Figure 3: Level 0 of project.....	6
Figure 4: HTML and CSS code layout.....	8
Figure 5: login screen.....	9
Figure 6: home page screen.....	9
Figure 7: documents screen.....	9
Figure 8: videos screen.....	10
Figure 9: create DB-1	10
Figure 10: create DB-2	11
Figure 11: create DB-3	11
Figure 12: main codes structure	12
Figure 13: server.py	13
Figure 14: danfoss_ZeroDay.py1	14
Figure 15: danfoss_ZeroDay.py2	14
Figure 16: test.....	15
Figure 17: test DB	15
Figure 18: create Heroku account.....	16
Figure 19: admin_dashboard_page.....	17
Figure 20: add_user	17
Figure 21: admin_dashboard_page_css.....	18
Figure 22: admin.py	18
Figure 23: danfoss_ZeroDay.py3	19
Figure 24: server.py2.....	20
Figure 25: multiple addition	20
Figure 26: users	21
Figure 27: admin_homepage	21
Figure 28: admin_requiredDocuments	22
Figure 29: admin_videopage	22
Figure 30: admin_dashboardPage	23
Figure 31: connect Heroku with GitHub	23
Figure 32: connect codes from GitHub	24
Figure 33: connect tables.....	24
Figure 33: insert data at tables.....	25
Figure 34: error.....	25
Figure 35: monitoring project.....	25
Figure 36: check db connection.....	26
Figure 37: Heroku Postgres utilization.....	27

Project Description

The main goal of my project is to help new employees start their first day at work smoothly. With this project, companies can show new employees all the important documents on a website before they come to the office. They can also watch safety videos and other important videos online. This way, the first day will be easier, and both the new employee and the company will save time during the hiring process.

The project will be built using Python Flask, Heroku Postgres, Bootstrap, and Blueprint. Python Flask will be used to create the web interface and manage all the logic of the app. Heroku Postgres will store and manage the data in the cloud, making it easier to handle backups and scale up if more users join.

Bootstrap will be used to create a user-friendly interface that works well on all devices, like phones, tablets, and computers. Blueprint will help organize the code into different sections, making it easier to manage and reuse. Together, these tools will make the project more efficient, flexible, and easy to use for everyone.

Cloud Delivery Models

SaaS (Software as a Service):

I will utilize various SaaS solutions to enhance my project functionalities, providing user-friendly services for specific needs without going into specific providers.

PaaS (Platform as a Service):

I will use Heroku to deploy my application because it is a Platform as a Service (PaaS) that makes it easy to develop, run, and manage my Python Flask application. Heroku has many benefits, like automatic scaling, which adds more resources when my application gets more visitors, ensuring it runs smoothly. It also supports continuous integration and deployment (CI/CD), which helps me update my app quickly and efficiently.

IaaS (Infrastructure as a Service):

I will use Heroku Postgres as my database solution. Heroku Postgres provides scalable database services that can easily integrate with my application, allowing me to manage and store data efficiently in the cloud.

Diagrams

Use Case Diagram

You can see the use case diagram at Figure 1.

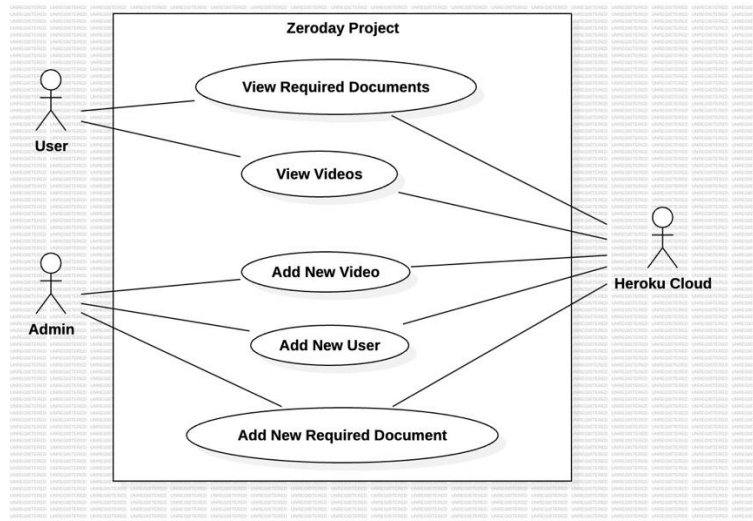


Figure 1: use case diagram of project

Data Flow Diagram

You can see the data flow diagram at Figure 2 and Figure 3.

Context Level:

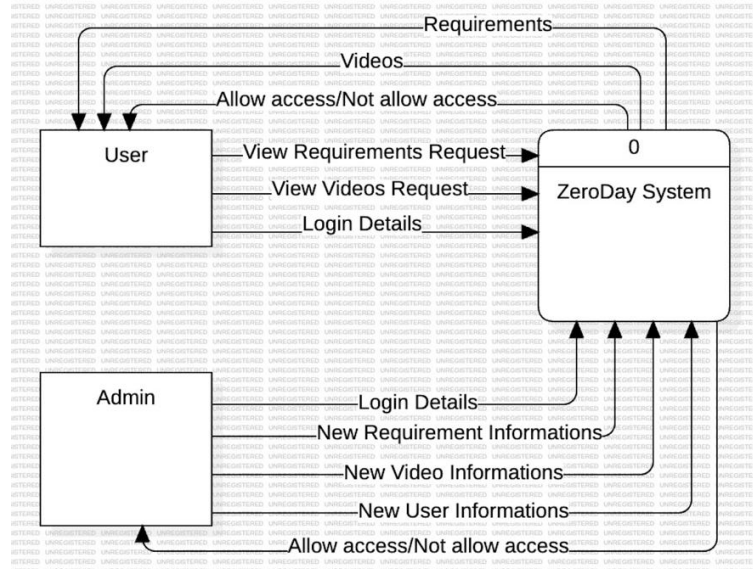


Figure 2: context level of project

Level 0:

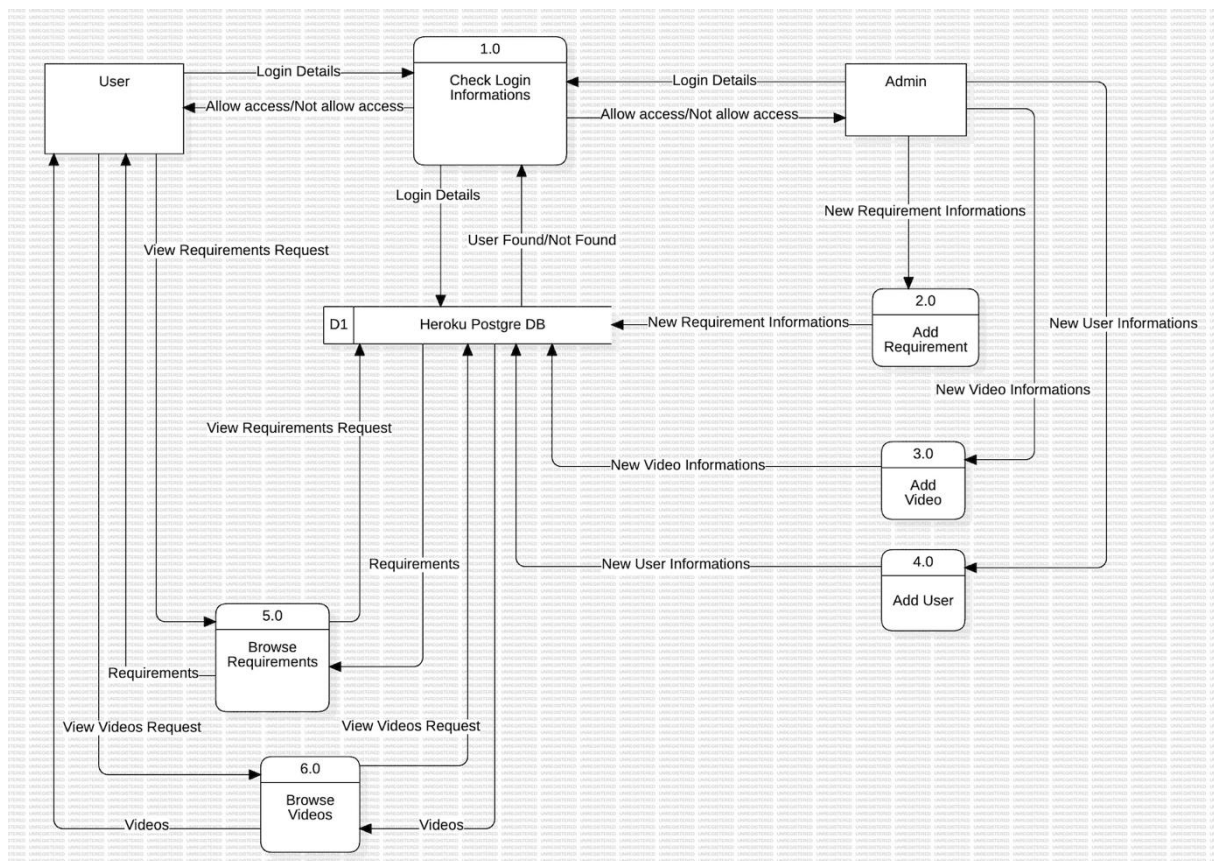


Figure 3: Level 0 of project

Data Types

In my project, the types of data I use are:

Text Data:

- **Video Names and Links:** In the "videos" section, I show the names and links of the videos that new employees can watch before their first day at work.
- **User Login Information:** Users log in to the system using a username and a password to access the platform.
- **Data Stored in the Database:** The database in my project stores important information such as usernames, passwords, video names, links, like/dislike counts, and required documents in text format.

Computation

In my project, computation involves processing data and performing calculations. For example:

- Checking user login information when users access the platform.
- Updating the like/dislike counts for each video when users interact with them.
- Adding new user (create username, password and decide he/she is admin or not).

Expected Contribution

The backend, frontend, database and cloud parts of the project will be done by Umutcan Celik.

Releated Work

1. SAP SuccessFactors Onboarding: This solution simplifies the onboarding process for new employees by integrating systems, processes, and people into a mobile-friendly and intuitive experience.[6]
2. Workday Onboarding: Workday's onboarding module helps new employees quickly adapt to company culture and complete necessary documentation, streamlining the hiring process. [7]
3. BambooHR Onboarding: Designed for small and medium-sized businesses, BambooHR offers a user-friendly onboarding process to help new employees prepare for their first day. [8]
4. Google Workspace Onboarding: Google Workspace facilitates access to company resources and collaboration for new employees, ensuring a smooth onboarding process. [9]

Novelties of the Project

- Pre-Onboarding Focus: Unlike traditional systems, the project prepares employees before their first day by providing access to key documents and videos online, saving time during onboarding.
- Cloud Accessibility: Hosting on Heroku and using Heroku Postgres ensures scalability, reliability, and easy access from anywhere.

- **Dynamic Content Management:** Admins can easily update requirements and videos, making the system adaptable to company-specific needs.
- **Modern Tech Stack:** Combines Python Flask, Bootstrap, and Flask Blueprints for a scalable, responsive, and modular design.

Contribution to the Field

The project modernizes onboarding by focusing on pre-arrival preparation, offering a scalable and user-friendly platform that saves time and enhances the onboarding process for both companies and employees.

Milestones Achieved

Week 4 (Oct 21 - 27) -- Week 5 (Oct 28 - Nov 3)

I completed my HTML and CSS codes. In this section, I added comments to keep the codes organized and readable and separated my CSS and HTML codes under static/css and templates files. You can see from Figure 4.

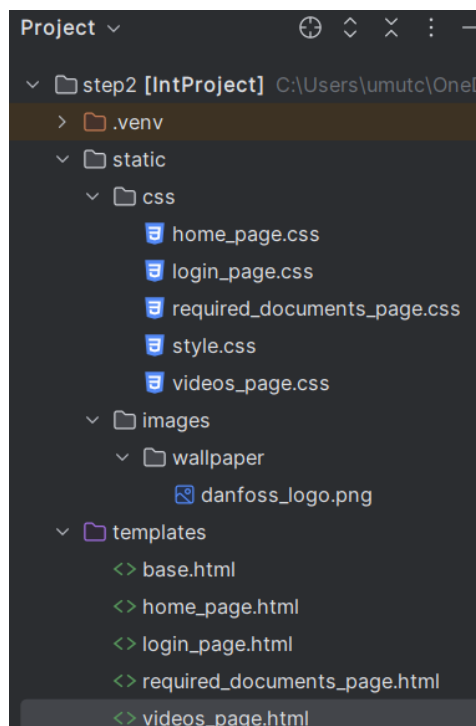


Figure 4: HTML and CSS code layout

You can see the interfaces created by the HTML and CSS codes I wrote from figure 5,6,7,8.

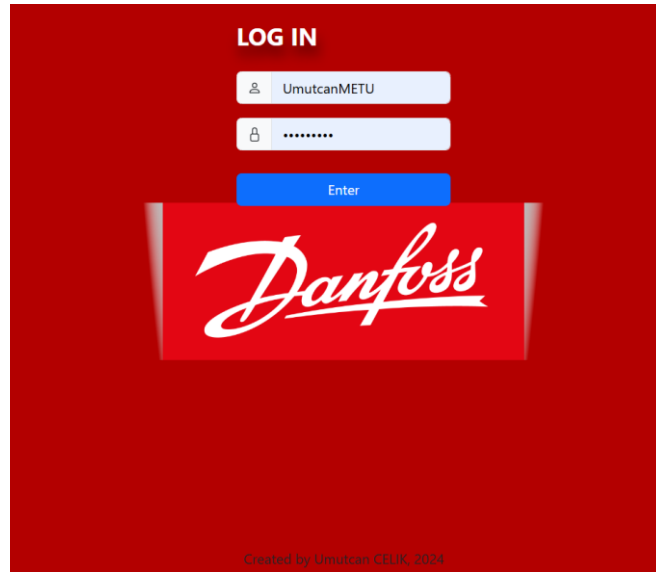


Figure 5: login screen

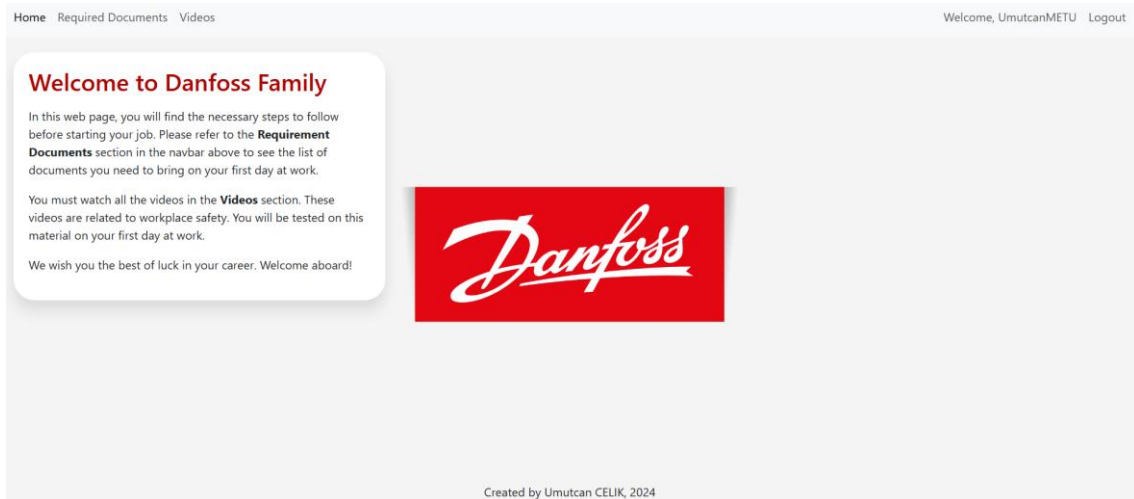


Figure 6: home page screen

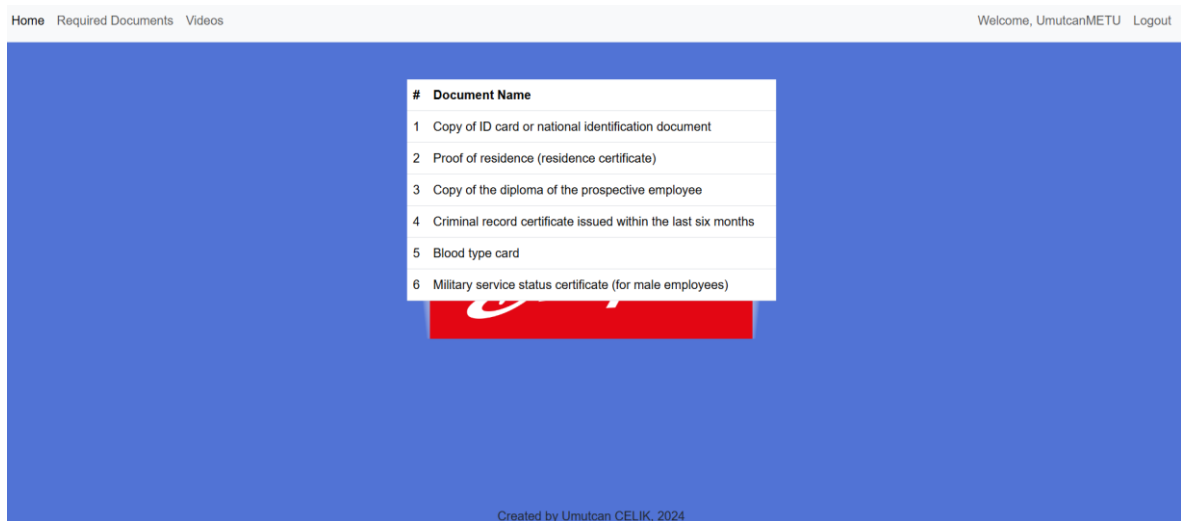


Figure 7: documents screen

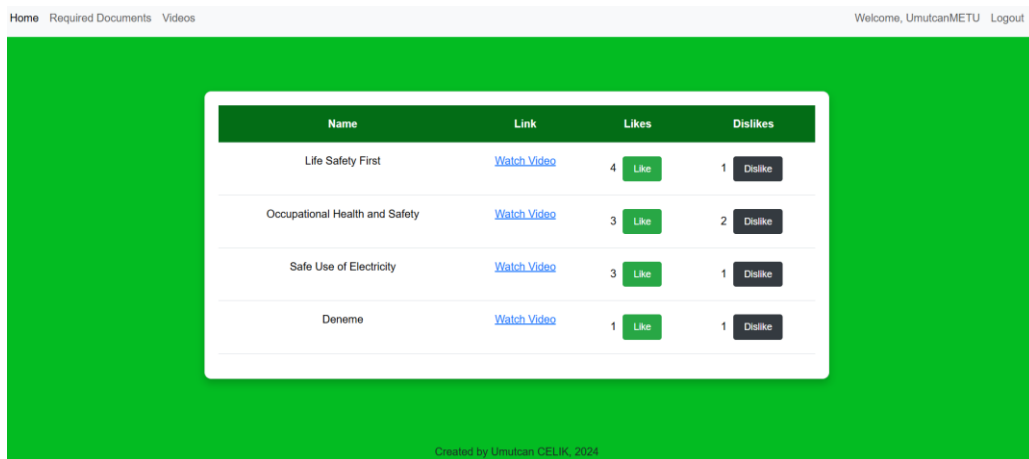


Figure 8: videos screen

Week 6 (Nov 4 - 10)

In these weeks, I designed the DB that I will use in PostgreSQL and added a small number of data in them. You can check my work on this from figure 9,10,11.

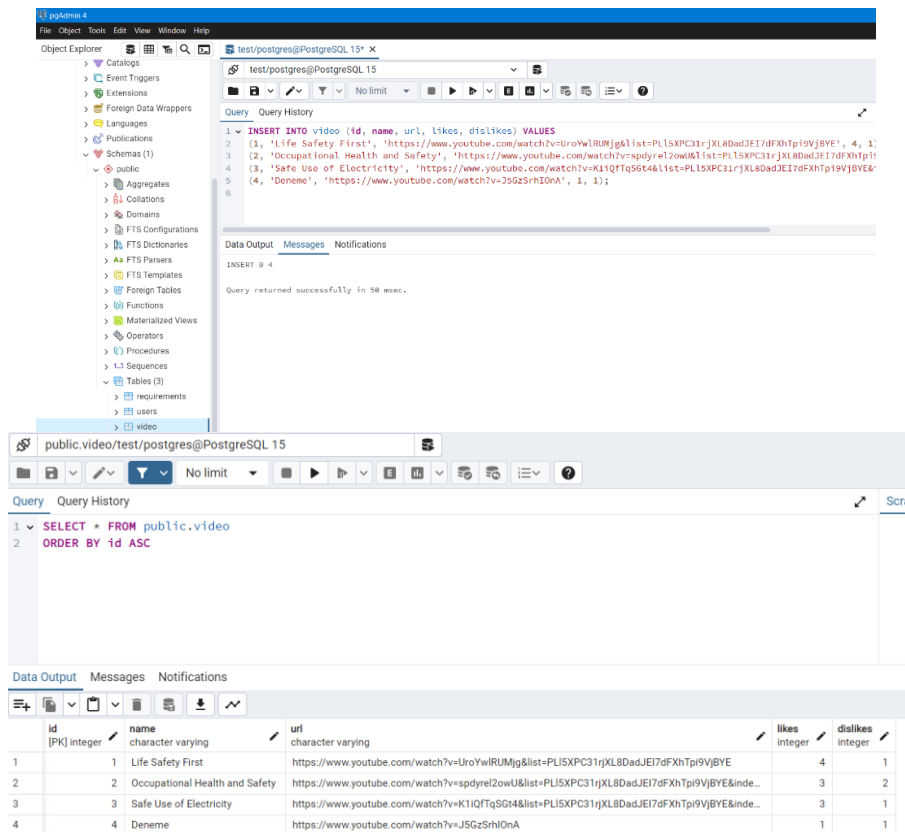
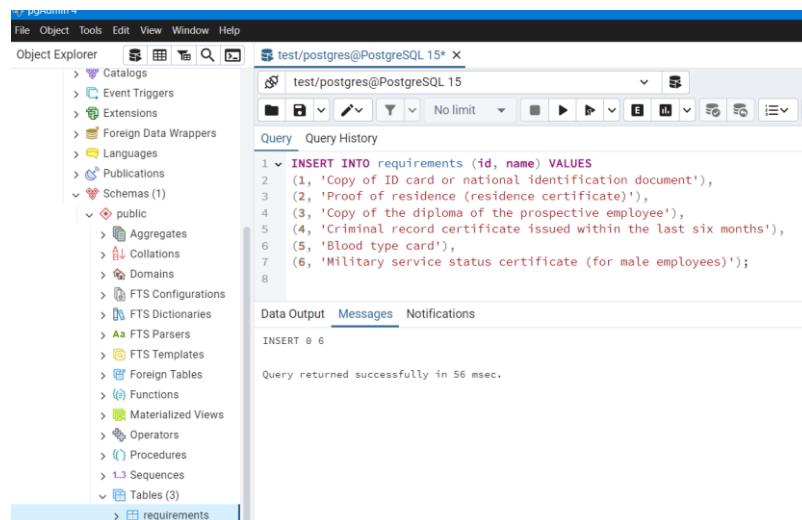
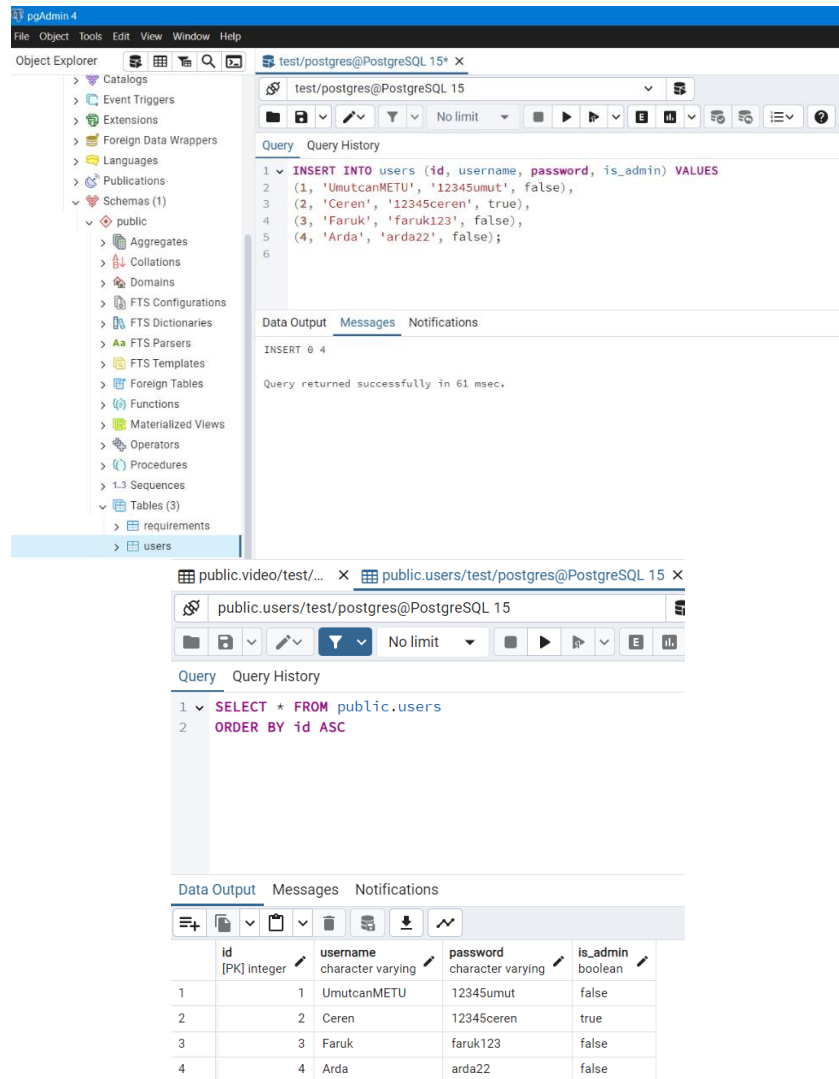


Figure 9: create DB-1



Week 7 (Nov 11 - 17) -- Week 8 (Nov 18 - 24)

Between these weeks I completed the main part of my code. You can see the file structure of my main code in figure 12. Since I am using Blueprint, I have added these codes by opening an extra file named views.

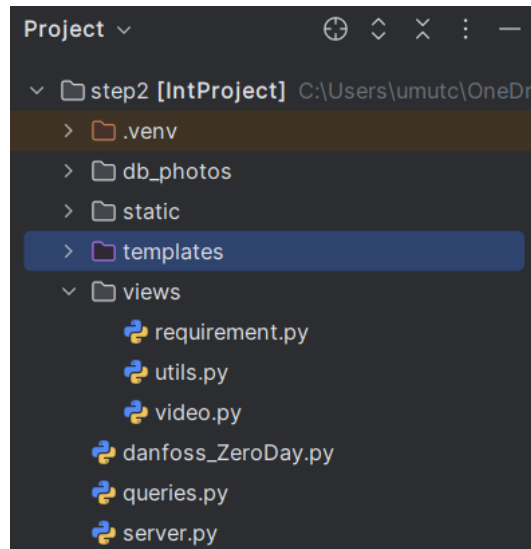


Figure 12: main codes structure

I want to talk a little bit about the important parts of my code. You can see my server.py code in Figure 13.

```
server.py x
1  from flask import Flask, redirect, url_for, render_template, request, session, flash
2  from psycopg2 import extensions
3  from queries import *
4
5  # Importing the requirement Blueprint
6  from danfoss_ZeroDay import initialize
7  from views.requirement import requirement
8  from views.utils import login_required
9  from views.video import video
10
11 # Register Unicode extensions for PostgreSQL
12 extensions.register_type(extensions.UNICODE)
13 extensions.register_type(extensions.UNICODEARRAY)
14
15 # /
16 app = Flask(__name__)
17 app.secret_key = 'METUNCC' # Secret key for session management
18
19 # Registering Blueprints for modular route handling
20 # Registering Blueprints for modular route handling
21 # /requirement
22 app.register_blueprint(requirement, url_prefix="/requirement")
23 # /videos
24 app.register_blueprint(video, url_prefix="/videos")
25
26 # Database configuration
27 HEROKU = False
28 if not HEROKU:
29     os.environ['DATABASE_URL'] = "dbname='test' user='postgres' host='localhost' password='umut62'"
30     initialize(os.environ.get('DATABASE_URL')) # Initialize the database connection
```

```

29 # Route for the home page, requires user to be logged in
30 <=
31 @app.route("/") 3 usages
32 @login_required # Ensure the user is logged in
33 <=
34 def home_page():
35     return render_template("home_page.html") # Render the home page template
36
37 # Route for the login page, handles GET and POST methods
38 <=login
39 @app.route("/login", methods=["GET", "POST"]) 6 usages
40 <=
41 def login():
42     if 'username' in session: # Check if the user is logged in
43         return redirect(url_for('home_page')) # Redirect to the home page
44
45     if request.method == "POST": # If the form is submitted
46         username = request.form.get("username") # Get username
47         password = request.form.get("password") # Get password
48
49         # Validate the input fields
50         if not username:
51             flash(message="Username cannot be empty.", category="warning")
52             return redirect(url_for('login')) # Redirect back to log in
53         if not password:
54             flash(message="Password cannot be empty.", category="warning")
55             return redirect(url_for('login')) # Redirect back to log in
56
57         # Query to get user information from the database
58         user = select(columns="username, password, is_admin", table="users", where=f"username='{username}'", as01ot=True)
59
60         # Check if the user exists and validate the password
61         if not user:
62             flash(message="No such username found.", category="danger")
63             return redirect(url_for('login')) # Redirect back to log in
64         elif user['password'] != password: # If the password is incorrect
65             flash(message="Incorrect password.", category="danger")
66             return redirect(url_for('login')) # Redirect back to log in
67         else:
68             # Set session variables for the logged-in user
69             session['username'] = user['username']
70             session['is_admin'] = user['is_admin'] # Add the admin status to the session
71             return redirect(url_for('home_page')) # Redirect to the home page
72
73     return render_template("login_page.html") # Render the login page template
74
75 # Route for logging out, requires user to be logged in
76 <=logout
77 @app.route("/logout") 1 usage
78 @login_required
79 def logout():
80     session.pop('username', None) # Remove the username from the session
81     return redirect(url_for('login'))
82
83 # Main entry point for the application
84 if __name__ == "__main__":
85     app.run(debug=True if not HEROKU else False) # Run the app in debug mode for local development

```

Figure 13: server.py

First, I imported necessary libraries for routing, templates, and database functions. Then, I set up **Blueprints** to keep things organized, like `/requirement` and `/videos` sections, which helps to manage routes separately. I also configured the PostgreSQL database connection for local development and prepared it for deployment on Heroku. The main parts include a **login system** where users enter a username and password, and I check if the info matches the database. If correct, the user can access the home page; if not, they get a warning. I also added a **logout** function to clear the session. Finally, the app runs in debug mode locally but will be turned off on Heroku. Next steps could involve adding admin features for more control over content.

Now I will talk about my `danfoss_ZeroDay.py` code where I manage my DB. In my code, I set up the database structure for my app using **PostgreSQL**. First, I imported libraries to work with environment variables and database connections. Then, I created some **SQL statements** that make important tables if they don't already exist. These include a user's table for storing usernames and passwords, a video table for videos with likes and dislikes, and a requirements table for important items. I also added an `is_admin` column to the users table to identify admin users. The initialize function runs these SQL statements when I connect to the database, setting up the tables automatically. I included a `get_all_requirements` function to get all entries from

the requirements table. Finally, I added a part at the end to make sure the database URL is set; if not, it gives a message and stops. Next steps might include adding more functions to manage data inside these tables more easily. You can examine my code from Figure 14 and 15.

```
danfoss_ZeroDay.py x
1  import os # For accessing environment variables
2  import sys # For system-specific parameters and functions
3  import psycopg2 as dbapi2 # PostgreSQL database interaction
4
5  # SQL statements to initialize the database tables
6  INIT_STATEMENTS = [
7      """
8      create table if not exists users(
9          id serial primary key,
10         username varchar not null unique,
11         password varchar not null
12     )
13     ,
14     """
15     create table if not exists video(
16         id serial primary key,
17         name varchar not null,
18         url varchar not null,
19         likes integer default 0 not null,
20         dislikes integer default 0 not null
21     )
22     ,
23     """
24     ALTER TABLE users ADD COLUMN IF NOT EXISTS is_admin BOOLEAN DEFAULT FALSE
25     ,
26     """
27     create table if not exists requirements(
28         id serial primary key,
29         name varchar not null
30     )
31     ,
32 ]
```

Figure 14: danfoss_ZeroDay.py1

```
34 # Initialize the database with the given URL
35 def initialize(url): 3 usages
36     with dbapi2.connect(url) as connection:
37         cursor = connection.cursor()
38         for statement in INIT_STATEMENTS:
39             cursor.execute(statement)
40         cursor.close()
41
42 # Function to get all requirements
43 def get_all_requirements(): 2 usages
44     """Retrieve all requirements from the requirements table."""
45     query = "SELECT id, name FROM requirements"
46     with dbapi2.connect(os.getenv("DATABASE_URL")) as connection:
47         cursor = connection.cursor()
48         cursor.execute(query)
49         requirements = cursor.fetchall()
50     return requirements
51
52 # Main execution to run the initialization
53 if __name__ == "__main__":
54     url = os.getenv("DATABASE_URL")
55     if url is None:
56         print("Usage: DATABASE_URL=url python danfoss_ZeroDay.py")
57         sys.exit(1)
58     initialize(url)
```

Figure 15: danfoss_ZeroDay.py2

While writing these codes, of course I also did tests, you can find these tests in figure 16.

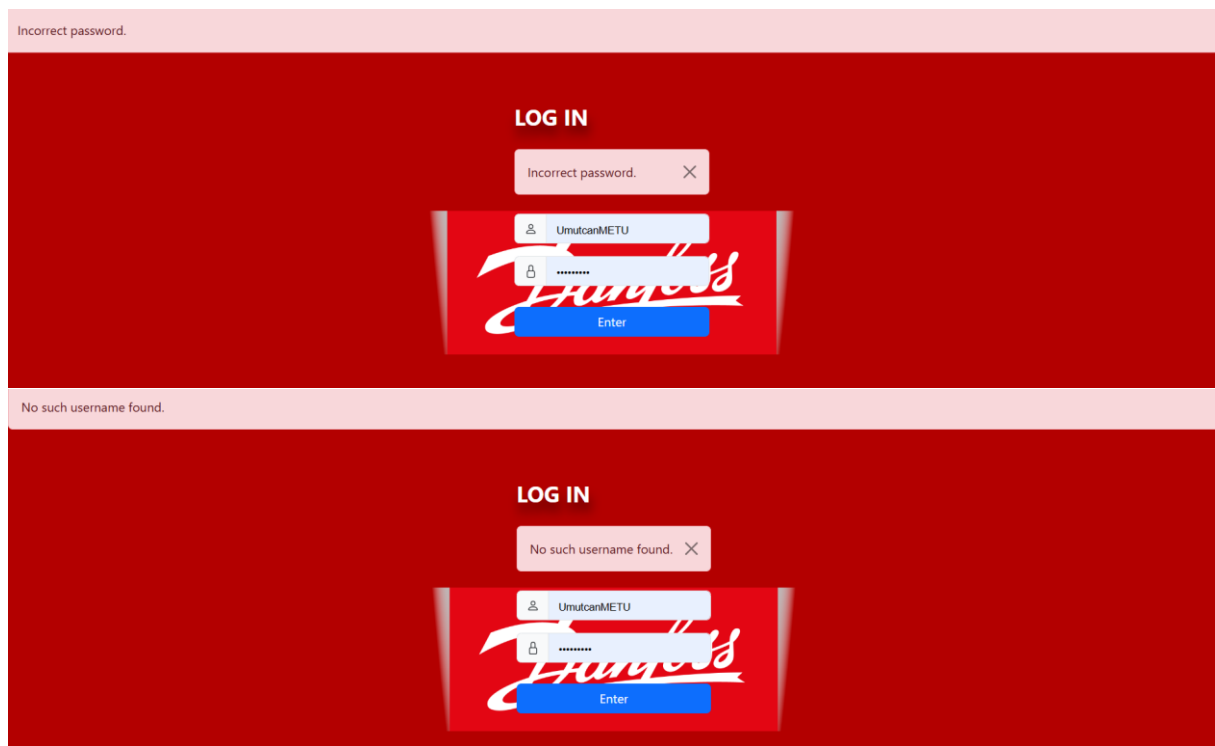


Figure 16: test

The test I did in Figure 17 was to check if the DB was updated for the videos.

The screenshot shows a web application interface with a green background. At the top, there is a navigation bar with links: Home, Required Documents, Videos. On the right, it says 'Welcome, UmutcanMETU' and 'Logout'. In the center, there is a table with the following data:

Name	Link	Likes	Dislikes
Life Safety First	Watch Video	4 <button>Like</button>	1 <button>Dislike</button>
Occupational Health and Safety	Watch Video	3 <button>Like</button>	2 <button>Dislike</button>
Safe Use of Electricity	Watch Video	3 <button>Like</button>	1 <button>Dislike</button>
Deneme	Watch Video	1 <button>Like</button>	1 <button>Dislike</button>

At the bottom, it says 'Created by Umutcan CELIK, 2024'.

Figure 17: test DB

After creating a Heroku account, I uploaded a test code and learned how to integrate it with postgresql. You can see this process in figure 18.

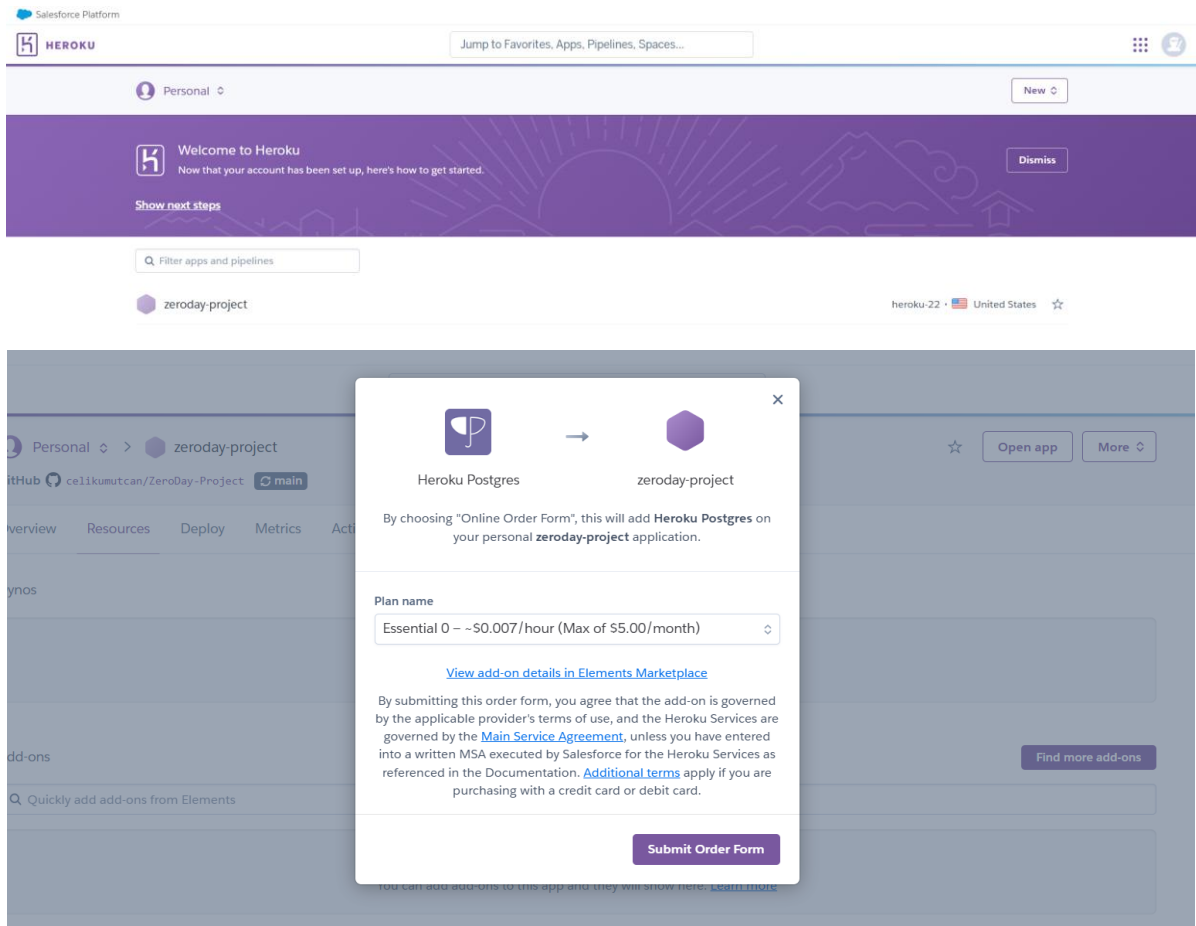


Figure 18: create Heroku account

The difficulties I had in my codes that I have completed so far were especially difficult to organize and connect the code accordingly, especially since I learned to use Blueprint from the beginning. Even though I didn't take a course on Python-Flask, creating this code took me time to research and learn. I made mistakes, but in the end I got a good result. For Heroku, I recommend using the recommended "Salesforce Authenticator" to avoid the "Authenticator" problem. I used "Microsoft Authenticator" and after a while I couldn't access the account. I contacted Help Support and after 2 days I was able to reactivate the account.

Week 9 (Nov 25 - Dec 1)

First, I created the HTML codes for the admin-specific interfaces. This code in figure 19, creates an admin dashboard page where administrators can view, add, and delete users with a responsive interface.


```

<> admin_dashboard_page.html x
1 <> {% extends "base.html" %}
2
3 {% block subtitle %}Admin Dashboard{% endblock subtitle %}
4
5 {% block content %}
6     <!-- Link to the admin dashboard stylesheet -->
7     <link rel="stylesheet" href="{% url_for('static', filename='css/admin_dashboard_page.css') %}">
8
9     <div class="container mt-5">
10         <div class="content-wrapper">
11             <div class="table-responsive">
12                 <table class="table">
13                     <thead>
14                         <tr>
15                             <!-- Index column -->
16                             <th>#</th>
17                             <!-- Column for the username -->
18                             <th>Username</th>
19                             <!-- Column for the password -->
20                             <th>Password</th>
21                             <!-- Column for admin status -->
22                             <th>Admin Status</th>
23                             <!-- Column for actions -->
24                             <th>Actions</th>
25                         </tr>
26                     </thead>
27                     <tbody>
28                         {% for user in users %}
29                             <tr>
30                                 <!-- Displays the current loop index -->
31                                 <td>{{ loop.index }}</td>
32                                 <!-- Displays the username -->

```

Figure 19: admin_dashboard_page

Next, I created the HTML code for adding new users. This code in figure 20 allows administrators to add a user by providing a username, password, and selecting whether the user has admin privileges through a simple form.

```

<> add_user.html x
1 <> {% extends "base.html" %}
2
3 {% block content %}
4     <!-- Header for the Add User section -->
5     <h2>Add User</h2>
6     <!-- Form to add a new user -->
7     <form method="POST">
8         <div class="mb-3">
9             <!-- Label for the username input -->
10            <label for="username" class="form-label">Username</label>
11            <!-- Input for username -->
12            <input type="text" class="form-control" id="username" name="username" required>
13        </div>
14        <div class="mb-3">
15            <!-- Label for the password input -->
16            <label for="password" class="form-label">Password</label>
17            <!-- Input for password -->
18            <input type="password" class="form-control" id="password" name="password" required>
19        </div>
20        <div class="form-check mb-3">
21            <!-- Checkbox for admin privileges -->
22            <input class="form-check-input" type="checkbox" id="is_admin" name="is_admin">
23            <!-- Label for the admin checkbox -->
24            <label class="form-check-label" for="is_admin">Is Admin</label>
25        </div>
26        <!-- Submit button to add the user -->
27        <button type="submit" class="btn btn-primary">Add User</button>
28    </form>
29 {% endblock content %}

```

Figure 20: add_user

After that, I designed the CSS for the admin dashboard page. This code in figure 21 customizes the page by setting a golden background, styling the container with rounded corners and shadows, and enhancing the table with a polished look, including red borders and bold headers for a clean and professional interface.

```

1  /* Sets the background color and font for the entire page */
2  body {
3      background-color: #cc8f00;
4      font-family: 'Arial', sans-serif;
5  }
6
7  /* Styles the container with background color, border radius, shadow, padding, and top margin */
8  .container {
9      background-color: rgb(202, 202, 202);
10     border-radius: 15px;
11     box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
12     padding: 30px;
13     margin-top: 50px;
14 }
15
16 /* Styles the table with border, border radius, overflow, and shadow */
17 .table {
18     border-collapse: separate;
19     border-spacing: 0;
20     border: 1px solid #d90000;
21     border-radius: 10px;
22     overflow: hidden;
23     box-shadow: 0 0 20px rgba(0, 0, 0, 0.1);
24 }
25
26 /* Adds padding and bottom border to table cells, aligns text to the left */
27 .table th,
28 .table td {
29     padding: 15px;
30     text-align: left;
31     border-bottom: 1px solid #ddd;
32 }

```

Figure 21: admin_dashboard_page_css

Then, I implemented the backend functionality for the admin panel using Flask Blueprints, enabling administrators to manage users through routes that display the user dashboard, handle the addition of new users via a form, and allow user deletion by ID, with feedback provided through flash messages. You can check from figure 22.

```

1  import ...
2
3  # Create the admin Blueprint with a specific URL prefix
4  <# admin
5
6  @admin = Blueprint(name='admin', __name__=__name__, template_folder='templates') # Prefix can be added during registration
7
8  # Renders the admin dashboard page with a list of users
9  <# admin/dashboard
10
11  @admin.route('/dashboard') @UMO
12  def dashboard():
13      users = get_all_users() # Fetches all users from the database
14      return render_template(template_name_or_list='admin/admin_dashboard_page.html', users=users)
15
16  # Handles adding a new user, accessible via GET and POST methods
17  <# admin/add_user
18
19  @admin.route('/add_user', methods=['GET', 'POST']) @UMO
20  def add_user_route():
21      if request.method == 'POST':
22          username = request.form.get('username')
23          password = request.form.get('password')
24          is_admin = request.form.get('is_admin') == 'on'
25
26          if username and password:
27              add_user(username, password, is_admin)
28              flash(message="User added successfully!", category="success")
29              return redirect(url_for('admin.dashboard'))
30          else:
31              flash(message="Please provide a valid username and password.", category="danger")
32      return render_template('admin/add_user.html')
33
34  # Deletes a user based on the provided user ID
35  <# admin/delete_user/user_id

```

Figure 22: admin.py

To enhance the admin features, I updated the database functionality by adding operations for managing users and requirements. Functions such as `add_user`, `get_all_users`, and `delete_user_by_id` were implemented to allow adding, viewing, and deleting users, with support for setting admin status. Additionally, I added `add_requirement_to_db`, `get_all_requirements`, and `delete_requirement_by_id` to provide admins the ability to dynamically manage the requirements list. These updates ensure that the admin panel is fully functional and integrated with the backend. You can check from figure 23.

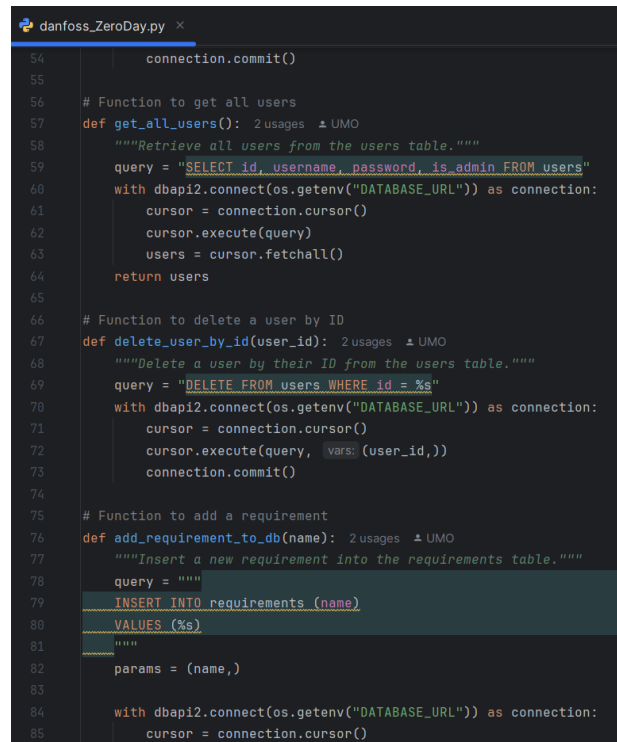
The image shows a code editor window titled 'danfoss_ZeroDay.py'. The code is Python and defines three functions: `get_all_users`, `delete_user_by_id`, and `add_requirement_to_db`.
Line 54: `connection.commit()`
Line 55: (blank)
Line 56: `# Function to get all users`
Line 57: `def get_all_users(): 2 usages 1 UMO`
Line 58: `"""Retrieve all users from the users table."""`
Line 59: `query = "SELECT id, username, password, is_admin FROM users"`
Line 60: `with dbapi2.connect(os.getenv("DATABASE_URL")) as connection:`
Line 61: `cursor = connection.cursor()`
Line 62: `cursor.execute(query)`
Line 63: `users = cursor.fetchall()`
Line 64: `return users`
Line 65: (blank)
Line 66: `# Function to delete a user by ID`
Line 67: `def delete_user_by_id(user_id): 2 usages 1 UMO`
Line 68: `"""Delete a user by their ID from the users table."""`
Line 69: `query = "DELETE FROM users WHERE id = %s"`
Line 70: `with dbapi2.connect(os.getenv("DATABASE_URL")) as connection:`
Line 71: `cursor = connection.cursor()`
Line 72: `cursor.execute(query, vars=(user_id,))`
Line 73: `connection.commit()`
Line 74: (blank)
Line 75: `# Function to add a requirement`
Line 76: `def add_requirement_to_db(name): 2 usages 1 UMO`
Line 77: `"""Insert a new requirement into the requirements table."""`
Line 78: `query = """`
Line 79: `INSERT INTO requirements (name)`
Line 80: `VALUES (%s)`
Line 81: `"""`
Line 82: `params = (name,)`
Line 83: (blank)
Line 84: `with dbapi2.connect(os.getenv("DATABASE_URL")) as connection:`
Line 85: `cursor = connection.cursor()`

Figure 23: danfoss_ZeroDay.py3

Finally, I updated the `server.py` file to integrate the admin features into the main application. I registered the admin Blueprint with the URL prefix `/admin` to modularly handle all admin-related routes. Additionally, I ensured that the database initialization (`initialize`) includes the required tables for managing admin functionalities. These adjustments allow seamless integration of the admin dashboard into the project, providing administrators with tools to manage users and requirements effectively. You can see from the figure below.

```

danfoss_ZeroDay.py  server.py x
def login():
    if request.method == "POST": # If the form is submitted
        username = request.form.get("username") # Get username
        password = request.form.get("password") # Get password

        # Validate the input fields
        if not username:
            flash(message="Username cannot be empty.", category="warning")
            return redirect(url_for('login')) # Redirect back to log in
        if not password:
            flash(message="Password cannot be empty.", category="warning")
            return redirect(url_for('login')) # Redirect back to log in

        # Query to get user information from the database
        user = select(column("username", password, is_admin), table="users", where=f"username='{username}'"), asDict=True)

        # Check if the user exists and validate the password
        if not user:
            flash(message="No such username found.", category="danger")
            return redirect(url_for('login')) # Redirect back to log in
        elif user['password'] != password: # If the password is incorrect
            flash(message="Incorrect password.", category="danger")
            return redirect(url_for('login')) # Redirect back to log in
        else:
            # Set session variables for the logged-in user
            session['username'] = user['username']
            session['is_admin'] = user['is_admin'] # Add the admin status to the session
            return redirect(url_for('home_page')) # Redirect to the home page

    return render_template("login_page.html") # Render the login page template

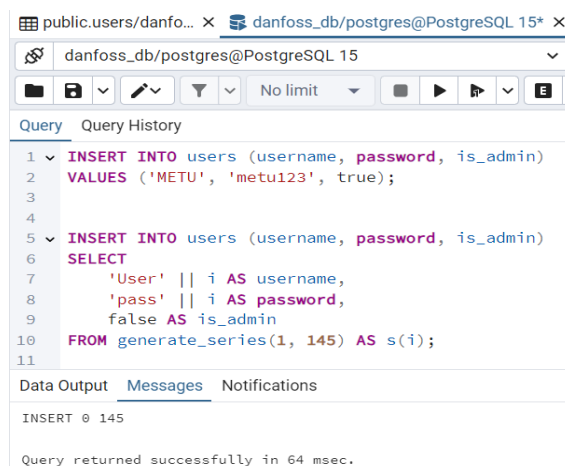
# Route for logging out, requires user to be logged in

```

Figure 24: server.py2

Week 10 (Dec 2 - 8)

That week, I expanded my system by adding 146 more users. You can see the SQL code for this in Figure 25. This allowed me to populate the users table with a total of 150 entries, including one additional admin user, "METU" By doing so, I ensured a more realistic database for testing the admin dashboard and user management functionalities. [5]



```

public.users/danfo...  danfoss_db/postgres@PostgreSQL 15* x
danfoss_db/postgres@PostgreSQL 15
Query  Query History
1  INSERT INTO users (username, password, is_admin)
2  VALUES ('METU', 'metu123', true);
3
4
5  INSERT INTO users (username, password, is_admin)
6  SELECT
7      'User' || i AS username,
8      'pass' || i AS password,
9      false AS is_admin
10 FROM generate_series(1, 145) AS s(i);
11
Data Output  Messages  Notifications
INSERT 0 145
Query returned successfully in 64 msec.

```

Figure 25: multiple addition

You can see from Figure 26 that the data was recorded successfully.

public.users/danfoss_db/postgres@PostgreSQL 15

public.users/danfoss_db/postgres@PostgreSQL 15

Query Query History

```
1 SELECT * FROM public.users
2 ORDER BY id ASC
```

Data Output Messages Notifications

	id [PK] integer	username character varying	password character varying	is_admin boolean
1	1	UmutcanMETU	12345umut	false
2	2	Ceren	12345ceren	true
3	5	Faruk	faruk123	false
4	7	Ardan	arda22	false
5	9	METU	metu123	true
6	10	User1	pass1	false
7	11	User2	pass2	false
8	12	User3	pass3	false
9	13	User4	pass4	false
10	14	User5	pass5	false
11	15	User6	pass6	false
12	16	User7	pass7	false
13	17	User8	pass8	false
14	18	User9	pass9	false
15	19	User10	pass10	false
16	20	User11	pass11	false
17	21	User12	pass12	false

Total rows: 150 of 150 Query complete 00:00:00.182 Ln 1, Col 1

Figure 26: users

If you want, let's look at the images of my updated project now. You can examine the homepage that opens when the admin logs into the system in Figure 27.

Home Required Documents Videos Admin Dashboard Welcome, Ceren Logout

Welcome to Danfoss Family

In this web page, you will find the necessary steps to follow before starting your job. Please refer to the **Requirement Documents** section in the navbar above to see the list of documents you need to bring on your first day at work.

You must watch all the videos in the **Videos** section. These videos are related to workplace safety. You will be tested on this material on your first day at work.

We wish you the best of luck in your career. Welcome aboard!



Created by Umutcan CELIK, 2024

Figure 27: admin_homepage

You can see from the Required Documents section in Figure 28 that the admin can add new requirements and delete old ones.

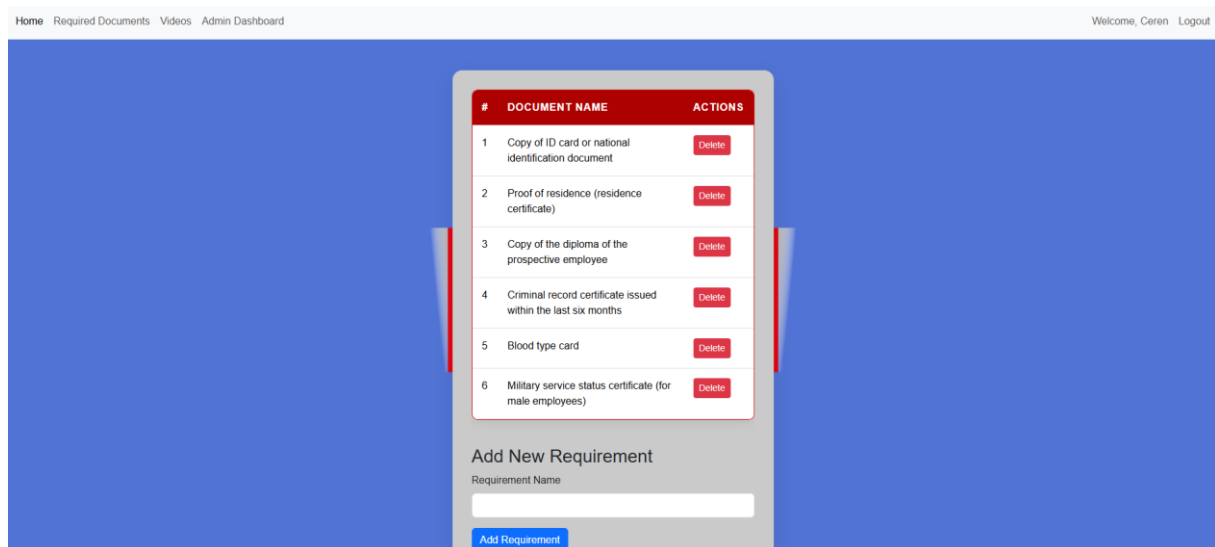


Figure 28: admin_requiredDocuments

From Figure 29 you can see that the admin can add and delete videos to the video page.

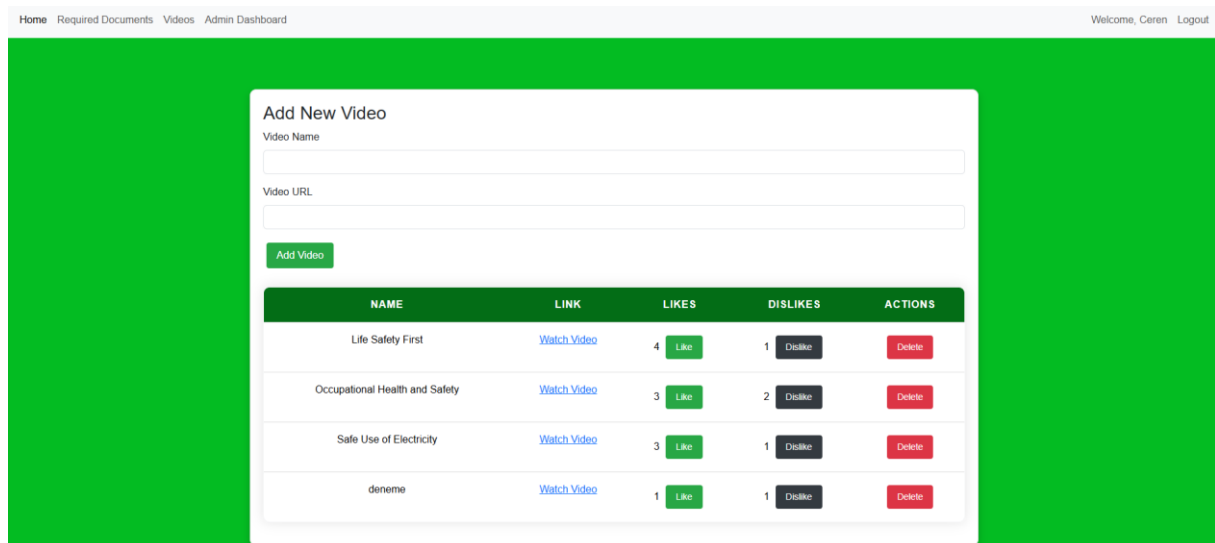


Figure 29: admin_videopage

And finally, you can see (from figure 30) that the admin can easily change users via the system without entering the DB. From here, they can add a new user and give permission to use them by giving them their username and password via email.

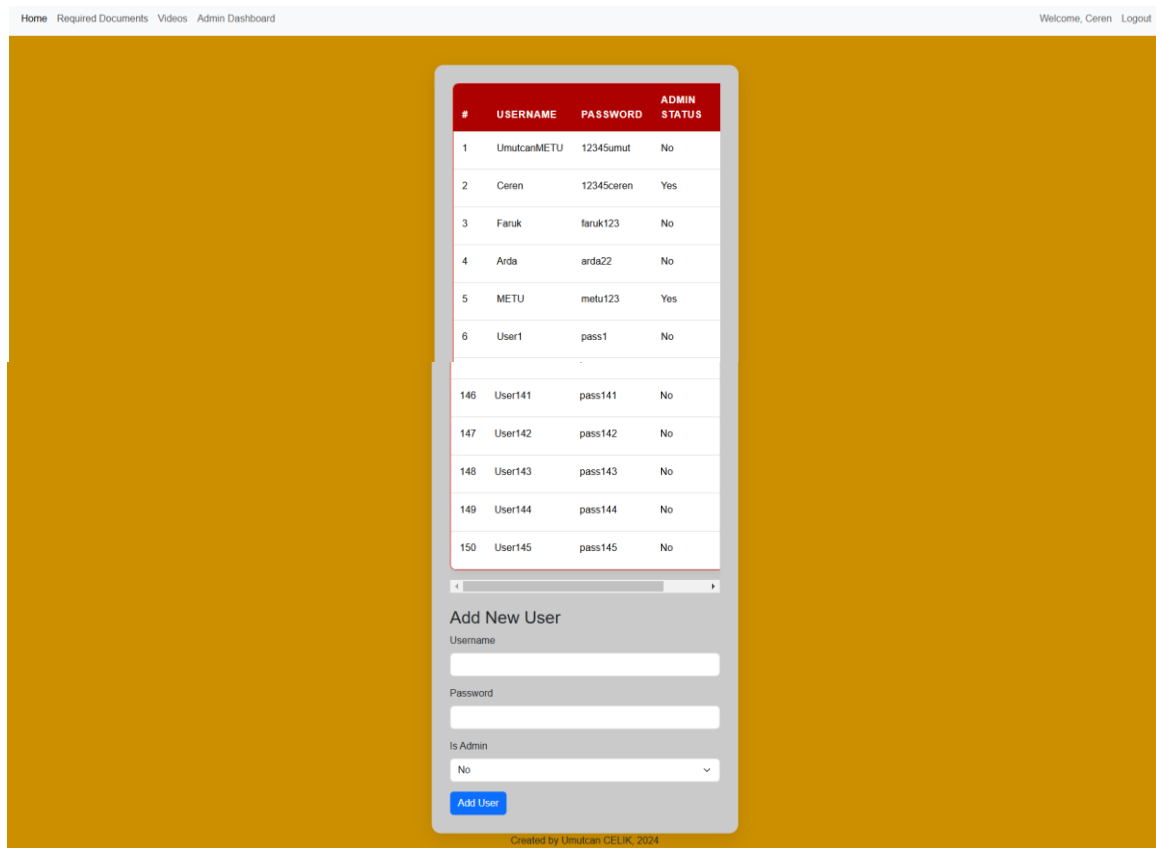


Figure 30: admin_dashboardPage

Week 11 (Dec 9 – Dec 15)

First of all, I connected the codes I uploaded to GitHub with my Heroku account. You can see this in figure 31.

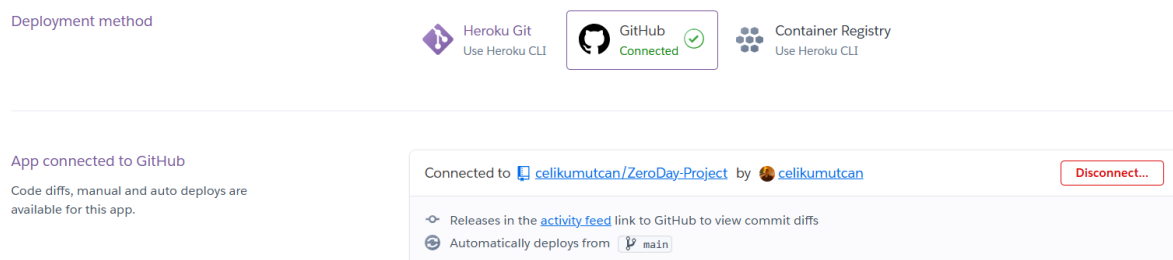


Figure 31: connect Heroku with GitHub

If you recall, we completed the connection with Heroku Postgres in figure 18. I am pulling our codes from GitHub to Heroku with Deploy Branch. You can see this in figure 32.

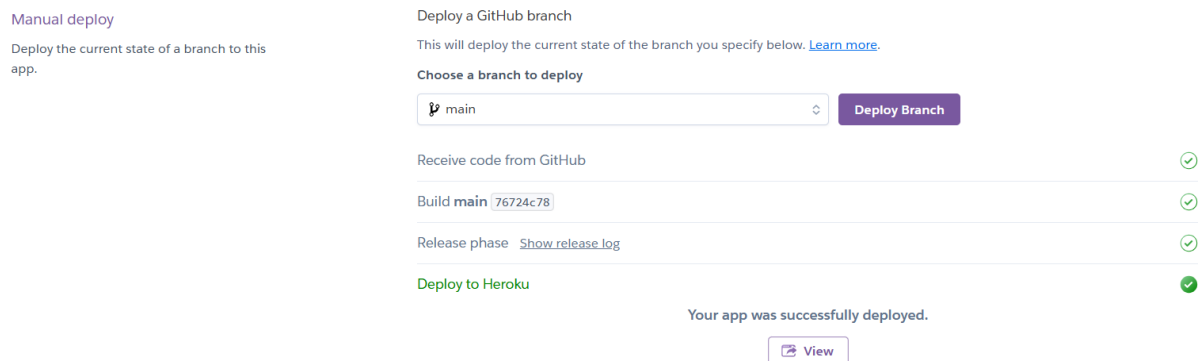


Figure 32: connect codes from GitHub

We move our tables to Heroku with the help of the code in Figure 33.

```
Microsoft Windows [Version 10.0.26100.2454]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\umutc\OneDrive - metu.edu.tr\Desktop\Final_Version>heroku psql -a zeroday-project
--> Connecting to postgresql-round-23962
psql (15.8, server 16.4)
WARNING: psql major version 15, server major version 16.
        Some psql features might not work.
WARNING: Console code page (857) differs from Windows code page (1254)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

zeroday-project::DATABASE=> \dt;
               List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | requirements | table | u8cm6dcsf2a3u5
 public | users      | table | u8cm6dcsf2a3u5
 public | video      | table | u8cm6dcsf2a3u5
(3 rows)

zeroday-project::DATABASE=>
```

Figure 33: connect tables

Then I added my insert codes. You can see it in Figure 34.


```

zeroday-project::DATABASE> INSERT INTO users (username, password, is_admin)
zeroday-project::DATABASE> SELECT
zeroday-project::DATABASE> 'User' || i AS username,
zeroday-project::DATABASE> 'pass' || i AS password,
zeroday-project::DATABASE> false AS is_admin
zeroday-project::DATABASE> FROM generate_series(1, 148) AS s(i);
INSERT 0 148
zeroday-project::DATABASE> INSERT INTO requirements (id, name) VALUES
zeroday-project::DATABASE> (1, 'Copy of ID card or national identification document'),
zeroday-project::DATABASE> (2, 'Proof of residence (residence certificate)'),
zeroday-project::DATABASE> (3, 'Copy of the diploma of the prospective employee'),
zeroday-project::DATABASE> (4, 'Criminal record certificate issued within the last six months'),
zeroday-project::DATABASE> (5, 'Blood type card'),
zeroday-project::DATABASE> (6, 'Military service status certificate (for male employees)');
INSERT 0 6
zeroday-project::DATABASE> INSERT INTO video (id, name, url, likes, dislikes) VALUES
zeroday-project::DATABASE> (1, 'Life Safety First', 'https://www.youtube.com/watch?v=UroYwIRUMjg&list=PLI5XPC31rjXL8DadJEI7dFXhTpI9VjBYE', 4, 1),
zeroday-project::DATABASE> (2, 'Occupational Health and Safety', 'https://www.youtube.com/watch?v=spdyrel2owU&list=PLI5XPC31rjXL8DadJEI7dFXhTpI9VjBYE&index=3', 3, 2),
zeroday-project::DATABASE> (3, 'Safe Use of Electricity', 'https://www.youtube.com/watch?v=KliQfTqSGt4&list=PLI5XPC31rjXL8DadJEI7dFXhTpI9VjBYE&index=8', 3, 3),
zeroday-project::DATABASE> (4, 'deneme', 'https://www.youtube.com/watch?v=J5GzSrh1OnA', 1, 1);
INSERT 0 4
zeroday-project::DATABASE>

```

Figure 33: insert data at tables

After adding the project to Heroku, I noticed that the requirements table was not updated. It was giving an error message. You can see this error in Figure 34.

Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

Figure 34: error

While trying to solve this problem, I checked if there were any errors with the code I used in Figure 35. I couldn't see any problems.

```

C:\Windows\System32\cmd.exe - heroku logs --tail --app zeroday-project
2024-12-12T14:27:22.718908+00:00 app[web.1]:
2024-12-12T14:27:22.718912+00:00 app[web.1]: Attempted Query
2024-12-12T14:27:22.718913+00:00 app[web.1]: SELECT id, name, url, likes, dislikes FROM video
2024-12-12T14:27:22.718913+00:00 app[web.1]: -----
2024-12-12T14:27:22.718914+00:00 app[web.1]:
2024-12-12T14:27:22.808539+00:00 app[web.1]: 10.1.21.206 - - [12/Dec/2024:14:27:22 +0000] "GET /videos/ HTTP/1.1" 200 95
58 "https://zeroday-project-9cc8a99830fd.herokuapp.com/requirement/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36"
2024-12-12T14:27:22.808746+00:00 heroku[router]: at=info method=GET path="/videos/" host=zeroday-project-9cc8a99830fd.h
herokuapp.com request_id=93de16c9-4036-4b73-aebc-7c5057a8b65d fwd="77.92.25.2" dyno=web.1 connect=0ms service=90ms status=
200 bytes=9727 protocol=https
2024-12-12T14:27:22.981205+00:00 app[web.1]: 10.1.21.206 - - [12/Dec/2024:14:27:22 +0000] "GET /static/css/style.css HTTP/1.1" 304 0 "https://zeroday-project-9cc8a99830fd.herokuapp.com/videos/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36"
2024-12-12T14:27:22.982299+00:00 heroku[router]: at=info method=GET path="/static/css/style.css" host=zeroday-project-9c
c8a99830fd.herokuapp.com request_id=dd854ea7-683b-42c4-a3c6-6984cc90f096 fwd="77.92.25.2" dyno=web.1 connect=0ms service=
2ms status=304 bytes=214 protocol=https
2024-12-12T14:27:22.988792+00:00 app[web.1]: 10.1.35.34 - - [12/Dec/2024:14:27:22 +0000] "GET /static/css/admin_dashboard
page.css HTTP/1.1" 304 0 "https://zeroday-project-9cc8a99830fd.herokuapp.com/videos/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36"
2024-12-12T14:27:22.989122+00:00 heroku[router]: at=info method=GET path="/static/css/admin_dashboard_page.css" host=zer
oday-project-9cc8a99830fd.herokuapp.com request_id=0e3996a0-2fa9-477f-bd85-7cfe155eda7a fwd="77.92.25.2" dyno=web.1 connect=1ms service=2ms status=304 bytes=229 protocol=https
2024-12-12T14:27:22.989981+00:00 app[web.1]: 10.1.8.142 - - [12/Dec/2024:14:27:22 +0000] "GET /static/css/videos_page.css HTTP/1.1" 304 0 "https://zeroday-project-9cc8a99830fd.herokuapp.com/videos/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36"
2024-12-12T14:27:22.990105+00:00 heroku[router]: at=info method=GET path="/static/css/videos_page.css" host=zeroday-proje
ct-9cc8a99830fd.herokuapp.com request_id=bb2a0375-1e97-4979-832a-a59d1f03d780 fwd="77.92.25.2" dyno=web.1 connect=0ms service=1ms status=304 bytes=220 protocol=https

```

Figure 35: monitoring project

Afterward, I navigated to the directory of my project and reconnected to the database. You can see the connection code in Figure 36. After reconnecting, the issue in my project was resolved.

The problem occurred because the database connection on Heroku was not active or correctly established during the deployment process. The required connection was re-established by reconnecting to the database using the provided Heroku PostgreSQL command, enabling the application to function properly.

```
C:\Users\umut\OneDrive - metu.edu.tr\Desktop\Final_Version>heroku pg:psql --app zeroday-project
--> Connecting to postgresql-round-23962
psql (15.8, server 16.4)
WARNING: psql major version 15, server major version 16.
Some psql features might not work.
WARNING: Console code page (857) differs from Windows code page (1254)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

zeroday-project::DATABASE=> \d requirements
              Table "public.requirements"
  Column |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
   id    | integer         |           | not null | nextval('requirements_id_seq'::regclass)
   name  | character varying |           | not null |
Indexes:
    "requirements_pkey" PRIMARY KEY, btree (id)

zeroday-project::DATABASE=>
```

Figure 36: check db connection

I deployed my project to Heroku, allowing me to host my database in the cloud (IaaS) and complete my platform development (PaaS). Since the project is accessible online, I also utilized SaaS services. You can access the details of my project from the links below.

Programming languages and line counts I used in my project:

Python – Flask: 397 lines

HTML: 439 lines

CSS: 171 lines

SQL: 300 lines

You can see in figure 37 what the paid Essential 0 package I use on Heroku Postgres offers me.

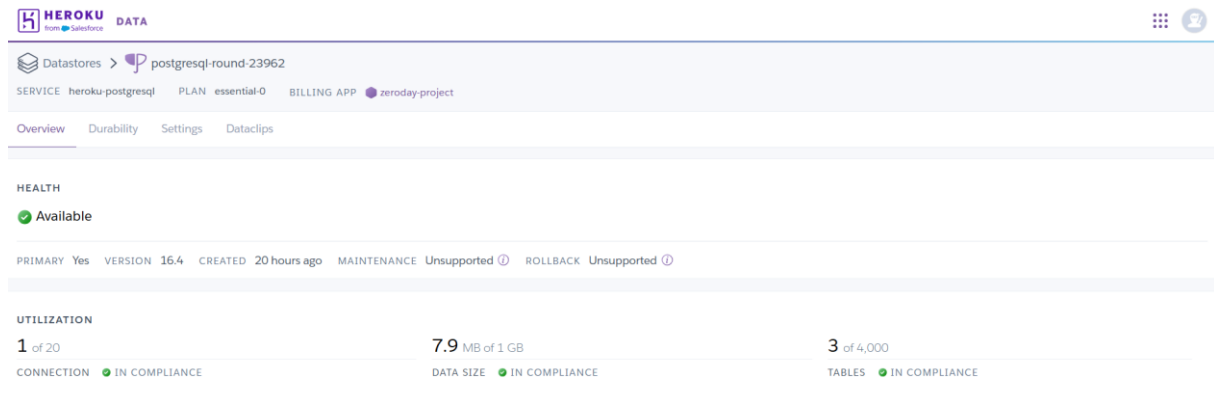


Figure 37: Heroku Postgres utilization

You can find the completed code for now on my github account:

- <https://github.com/celikumutcan/ZeroDay-Project>
- Github username: celikumutcan

You can also access my working project from the link below:

- <https://zeroday-project-9cc8a99830fd.herokuapp.com/>

References

1. Platform as a service | Heroku. (n.d.).
<https://www.heroku.com/platform>
2. Fully Managed database as a service - PostgreSQL | Heroku. (n.d.).
<https://www.heroku.com/postgres>
3. Data security in cloud computing using AES under HEROKU cloud. (2018, April 1). IEEE Conference Publication | IEEE Xplore.
<https://ieeexplore.ieee.org/abstract/document/8372705>
4. Andersson, N., & Chernov, A. (2016). Increasing the throughput of a Node.js application : running on the Heroku Cloud App platform. DIVA.
<https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A954978&dswid=5210>
5. Booz, R. (2024, September 6). How to Create (Lots of) Sample Time-Series Data With PostgreSQL generate_series(). Timescale Blog.
https://www.timescale.com/blog/how-to-create-lots-of-sample-time-series-data-with-postgresql-generate-series/?utm_source=chatgpt.com
6. SAP SuccessFactors Onboarding | Talent acquisition software. (n.d.). SAP.
<https://www.sap.com/products/hcm/employee-onboarding.html>
7. Workday Platform | HR, Finance, Planning, Spend. (n.d.).
<https://www.workday.com/>
8. HR Onboarding software for new employees | Create Stellar First Days | BambooHR. (n.d.).
<https://www.bamboohr.com/hr-software/employee-self-onboarding>
9. Workspace, G. (n.d.). Google Workspace | Business Apps & Collaboration Tools [Video]. Google Workspace.
<https://workspace.google.com/>